Log Analytics and Resolution

System Design

Lynn Hoang

Problem Statement

- Desktop app deployed across enterprise endpoints
- No direct access to customer machines for troubleshooting
- Need to automate troubleshooting at scale

Goals

- Solution works for any app
- Smart log collection
- Smart diagnostics for unknown failures
- Provide self-service resolutions
    - Self-service for enterprise admins, not auto-remediation
    - System provides actionable resolution; admin approves and coordinates rollout

Users

User	Role
Enterprise Admins	Support their organization’s users
Support Agents	Support enterprises who use products
Engineers	Develop products, analyze issues, deploy fixes

System Components

Logs Agent Client

- Collects, pre-processes, and sends logs to backend services for analysis

Backend Service

- Ingests, processes, and analyzes logs to identify issues
- Create tickets, request clients for more logs, generate resolutions
- Notify enterprise admins on resolution

Portal

- Enterprise admins view their own issues, status, dashboard
- Support agents and engineers view issues of all enterprises and internal data

Functional Requirements

Logs Agent Client

- Detect issues
- Collect relevant logs, classify issues (crash vs. hang vs. general issues)
- Collect additional logs as requested by Backend Service
- Scrub PII
- Send logs to Backend Service

Backend Service

- Validate and normalize logs
- Orchestrate log processing pipeline: analyze logs, create issue tickets, provide solutions or ask for additional logs
- Send notification on issues resolved or configs updated
- Service to Portal

Portal

- Allow Enterprise Admins to manage their issues and configurations and push changes to their users.
- Allow Support Agents/Engineers to manage all issues and internal data.

Non-Functional Requirements

Logs Agent Client

- Prioritize sending crash/hang logs over general issue logs
- Queue logs locally when offline
- Support resumable uploads
- Log collection must not degrade system performance

Backend Service

- Authentication/Authorization
  - Only authenticated users are allowed to access data they are authorized to view.
- Scalability: handle burst traffic (e.g., on app or OS updates)
- Strong consistency: Need all logs before debugging
- Security: Protection against abuse

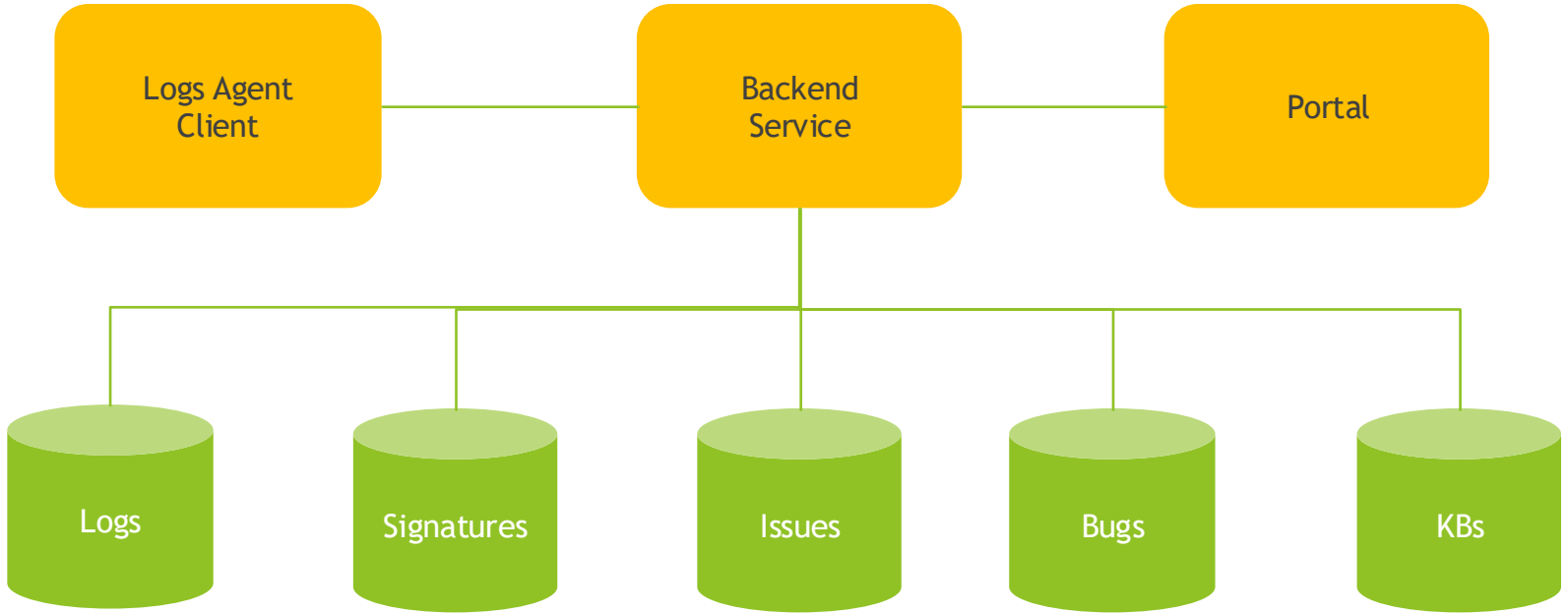
Portal

- High availability
- Fast dashboard and search response time

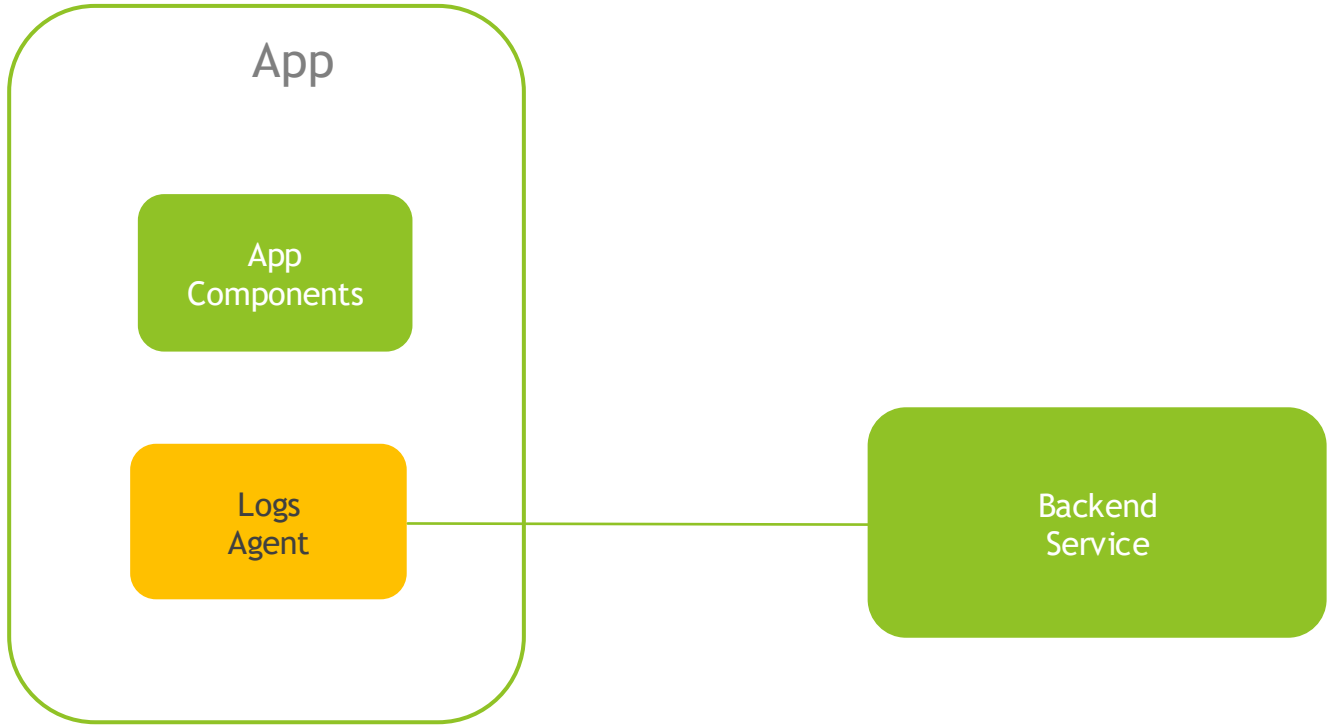
Scale Considerations

- 100K endpoints × 5 logs/day = 500K logs/day
- Burst: OS update could 10x traffic in 1 hour
- Queue absorbs spikes, auto-scaling backend workers

High-Level Architecture



Logs Agent Client



Logs Agent Client: Smart Log Collection

Key Design Decision: Configuration-driven (works for any app)

- Defines what app to monitor, what to collect, when to send, how to collect additional data (e.g., check AV, FW installed, VPN on/off status, network traffic)
- Ship with default configs: Collect crash, hang, app logs, app & system metadata
- On startup and on server notification, download latest configs from server
- Monitors: crash, hang, general issues

Benefits

- Allow customize log collection per app and enterprise environment
- Allow different configs per issue type
- Update configs without client update

Logs Agent Client: Configurations Sample

```
{
  "appName": "bestvpn",
  "productIdentifier": "bestvpn",
  ”includeTelemetry": true,
  "monitoringInterval": {
    "crash": {
      "frequency": "immediately"
    },
    "hang": {
      "frequency": "immediately"
    },
    "general": {
      "frequency": "interval",
      "seconds": 300
    }
  }
```

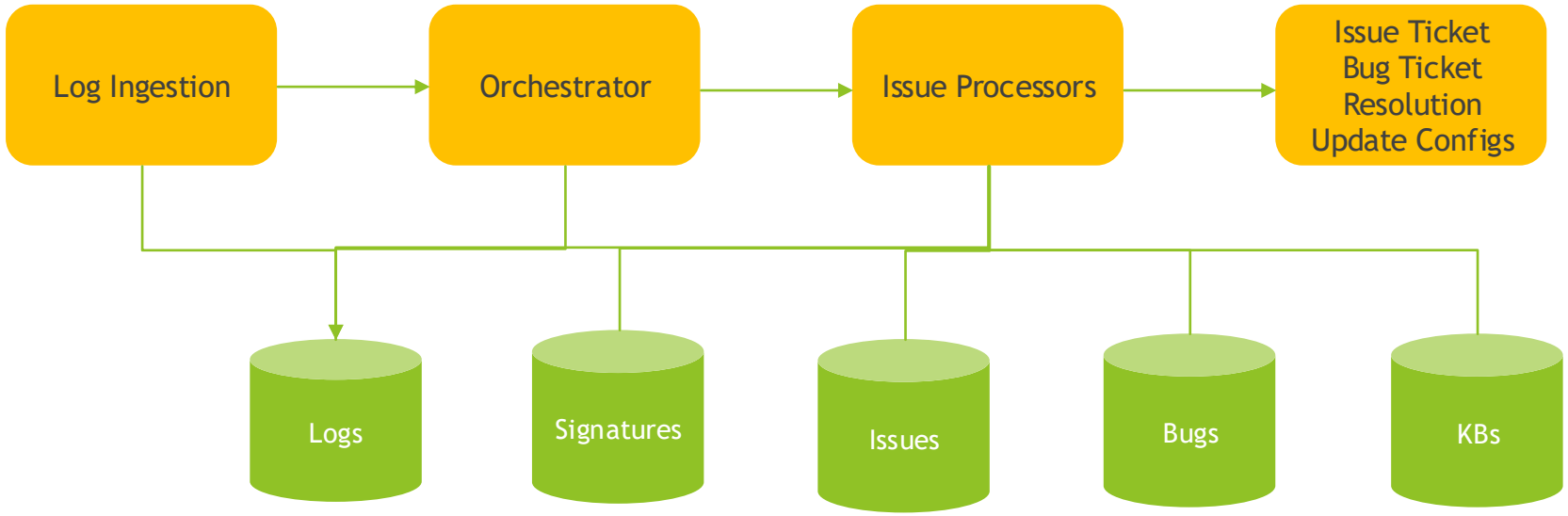
```
{
  "additionalCommands": [
    {
      "name": "download_speed",
      "command": "/usr/bin/curl",
      "arguments": [
        "%{speed_download}",
        "https://speed.cloudflare.com"
      ],
      "includeWithCrash": false,
      "includeWithHang": true
    },
    {
      "name": "upload_speed",
      "command": "/usr/bin/curl",
      "arguments": [
        "-X",
        "POST",
        "https://speed.cloudflare.com/__up"
      ],
      "includeWithCrash": false,
      "includeWithHang": true
    }
  ]
}
```

Logs Agent Client: Production-Readiness Requirements

Add

- PII scrubbing
- Support uploading large files in chunks (e.g., crash dumps)
- Support resume uploading

Backend Service: Log Processing Pipeline



- Handle burst traffic with queue
- Validate & normalize incoming logs
- Route by type (crash/hang/general)
- Fuzzy match against known signatures
- If match found + has resolution → auto-resolve → notify enterprise admins
- If no match → create new issue ticket, bug ticket

Backend Service: Smart Diagnostic

Key: Signature matching for resolution

- Engineers create signatures for known crash, hang, or general issues.
- Logs are matched against signatures to find known resolutions
- Fuzzy matching handles slight variations

Trade-off: Signature matching vs ML-based resolution

- Signature matching is interpretable — engineers see exactly why an issue matched
- Controllable — add/edit/remove signatures without retraining
- No cold start — works day one without labeled training data
- Future: ML could catch unknown patterns once we have labeled data from resolved issues

Backend Service: Smart Diagnostic (Continued)

For Unknown Issues,

- Engineer analyzes and determines what additional data is needed
- Engineer updates diagnostic configs → notifies enterprise admin → pushes to logs agent → logs agent collects
- Enables rapid iteration

Backend Service: Support Portal

Support CRUD operations of

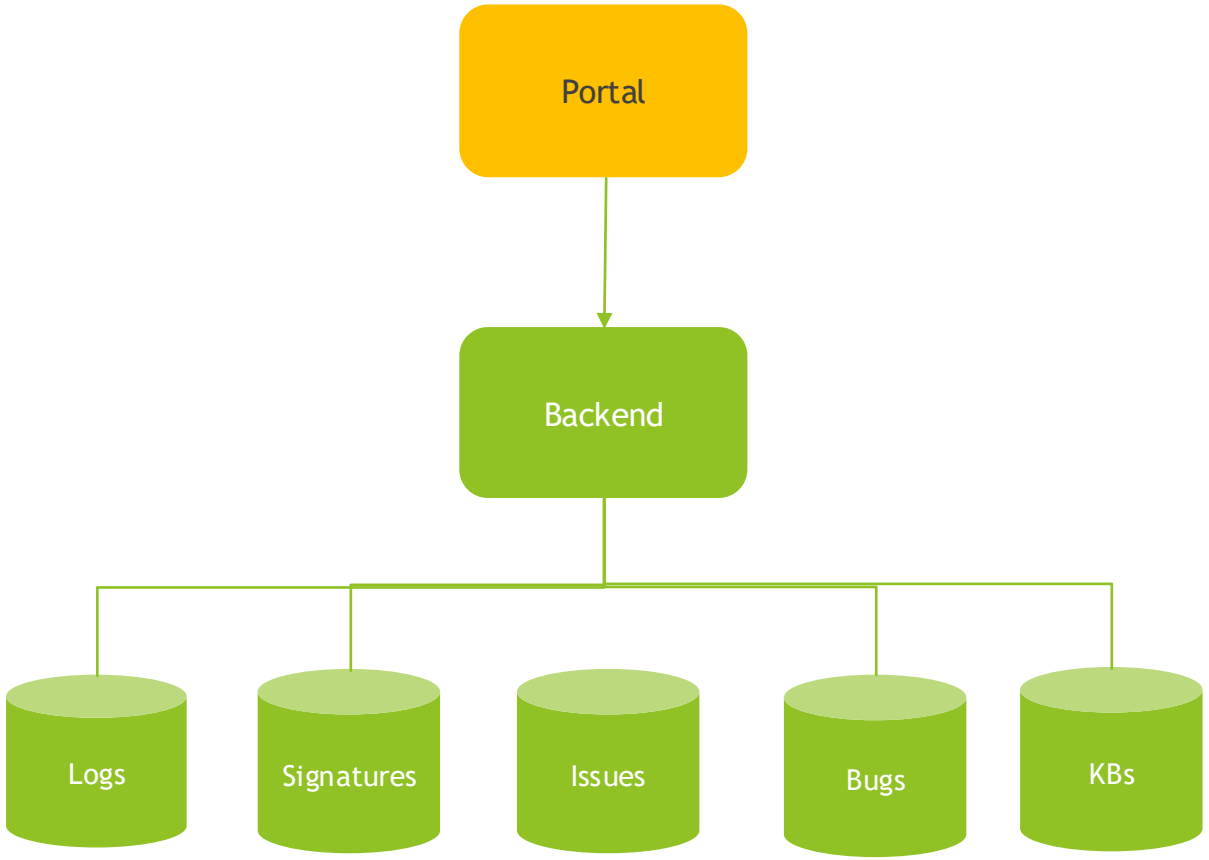
- Issues
- Signatures
- Diagnostic Configs

Backend Service: Production-Readiness Requirements

Add

- Deduplication: Group similar issues into a bucket
- Have rollback strategy for log configs
- Security: Protection against attack
- Fast access: Cache hot issues in memory
- Availability: horizontal scaling on burst traffics

Portal



Portal: Role-Based Access

- **Enterprise Admins**
  - View own organization's issues only
  - Can push log collection configs update to endpoints
  - Not allow to view internal details
- **Support Agents**
  - View issues, signatures, and log collection configs of all enterprises
- **Engineers**
  - Has all capabilities of Support Agents
  - Manage signatures of crash, hang, general issues and their resolutions
  - Manage log collection configs

API Design

Client → Backend

- POST /logs - Submit log payload
- GET /configs/:tenantId - Download diagnostic configuration

Portal → Backend

- /issues - CRUD + batch resolve matching
- /signatures - CRUD operations
- /configs - CRUD + deployment status

Key patterns

- RESTful design
- SSE for real-time log collection configs push to endpoints
- Role-based filtering on responses

System Quality

- Unit + integration tests (100% new code)
- Load testing for burst scenarios
- Signature matching accuracy metrics
- Alerting on pipeline latency

What I'd Add in Production

- AWS infrastructure
  - Portal domain: Route53
  - Portal responsiveness: CloudFront Distribution
  - Security: Rate limit, WAF
  - Backend services access: API Gateway, Proxy Lambdas
  - Backend microservices: Lambdas
  - Logs requests queue: SQS
  - Logs processing pipeline orchestration: Step Functions
  - DB storage: S3 (crash dumps), DynamoDB or PostgreSQL
  - Authentication: Secrets Manager, IAM, Cognito
  - Observability: CloudWatch, CloudTrail, X-Ray
- CI/CD pipeline
- Data retention & compliance

Demo

Thank You